

Lecture 07 Worksheet - Linalg-on-Tensors

목표: linalg.generic MatMul/Conv 를 직접 읽고 작성하며, NPU compiler lowering 관점에서 tiling/fusion 후보를 식별한다.

Part A. Operation Anatomy Annotation

아래 IR 에서 1) indexing_maps, 2) iterator_types, 3) ins/outs, 4) region arguments, 5) linalg.yield 가 의미하는 것을 주석으로 표시하세요.

```
#matmul_trait = {
  indexing_maps = [
    affine_map<(m, n, k) -> (m, k)>,
    affine_map<(m, n, k) -> (k, n)>,
    affine_map<(m, n, k) -> (m, n)>
  ],
  iterator_types = ["parallel", "parallel", "reduction"]
}

func.func @matmul_generic(%A: tensor<8x16xf32>,
                          %B: tensor<16x32xf32>) -> tensor<8x32xf32> {
  %c0 = arith.constant 0.0 : f32
  %init = tensor.empty() : tensor<8x32xf32>
  %zero = linalg.fill ins(%c0 : f32)
                                outs(%init : tensor<8x32xf32>) -> tensor<8x32xf32>
  %C = linalg.generic #matmul_trait
    ins(%A, %B : tensor<8x16xf32>, tensor<16x32xf32>)
    outs(%zero : tensor<8x32xf32>) {
    ^bb0(%a: f32, %b: f32, %c: f32):
      %p = arith.mulf %a, %b : f32
      %s = arith.addf %c, %p : f32
      linalg.yield %s : f32
    } -> tensor<8x32xf32>
  return %C : tensor<8x32xf32>
}
```

작성 공간:

Part B. Def-Use / Shape Table

Value	Type/Shape	Defined by	Used by	NPU 의미
%A				
%B				
%zero				
%C				

Part C. MatMul Generic 직접 작성

M=64, N=128, K=256 인 MatMul 을 tensor-based linalg.generic 으로 작성하세요. iterator_types 와 indexing_maps 를 반드시 포함하세요.

Part D. Conv NHWC/HWCF Map 작성

Input N,H,W,C = 1,30,30,16 / Filter KH,KW,C,F = 3,3,16,32 / Output = 1,28,28,32. loop domain 과 indexing_maps 를 작성하세요.

Operand	Indexing map	설명
Input NHWC		
Filter HWCF		
Output NHWC		

Part E. NPU Tile Footprint 계산

Output tile: OH=8, OW=8, F=16, reduction tile C=16, KH=3, KW=3 일 때 필요한 input/filter/output tile element 수 를 계산하세요. padding/stride 는 없다고 가정합니다.

Tile	Element count	Byte count FP16	SRAM placement
Input halo tile			
Filter tile			
Output/Accumulator tile			

Part F. Reflection

Linalg 단계에서 해결해야 하는 결정과 Bufferization 이후에 해결해도 되는 결정을 분리해 적으세요.
